

Job Intelligence System & Analytics Overview

1. Company Product & Operations

The core product is the **Job Intelligence Scraper** (part of the AllyNerds ecosystem). It is a robust, production-grade backend system designed to collect, process, deduplicate, and analyze job listings. **Operations involve:**

- Sourcing job descriptions via APIs (like SerpAPI or Exa API).
- Transforming and normalizing the raw data.
- Extracting key entities (like skills, experience levels, and compensation).
- Storing structured intelligence in a PostgreSQL database for reporting and candidate matching.

2. Core Technologies

- **PyCharm:** A professional Integrated Development Environment (IDE) used for Python development. It provides tools for debugging, virtual environments, and seamless database integration.
- **PostgreSQL (Postgres):** An advanced, open-source relational database. In this pipeline, it is used with strict schemas, constraints (Primary/Foreign keys, UNIQUE), and atomic transactions to ensure data integrity.
- **Antigravity (AGY):** The advanced AI coding assistant environment (your AI partner). It accelerates development by automating coding, running terminal commands, and managing database interactions intelligently.

3. The ETL Pipeline (Scrape -> Transform -> Store)

ETL stands for Extract, Transform, Load. Your pipeline follows this pattern:

- **Scrape (Extract):** Fetching raw job listings from external sources using APIs. The original scraped JSON/HTML is often saved in a `job_raw_data` table for auditing and reprocessing.
- **Transform:** The most crucial step where unstructured data becomes intelligence.
 - **Data Cleaning & Normalization:** Standardizing locations, salaries, dates, and company names.
 - **Deduplication:** Ensuring identical jobs from different sources aren't duplicated (e.g., using `UNIQUE(job_url)`).
 - **Skill Extraction:** Parsing the job description to identify specific technologies (e.g., Python, PostgreSQL, Docker) required for the job.
- **Store (Load):** Inserting the normalized data into relational database tables (`companies`, `jobs`, `skills`, `job_skills`) using database transactions (`BEGIN`, `COMMIT`, `ROLLBACK`) to guarantee that either all related data is saved safely, or none of it is, preventing partial writes.

4. NLP & Data Extraction Tools

To extract structured intelligence from unstructured job descriptions, we use several text processing techniques:

- **Regex (Regular Expressions):** A powerful syntax for matching specific text patterns. Used for straightforward tasks like extracting salary ranges (`\$[0-9]{2,3}k`) or finding exact keyword matches.
- **spaCy:** An industrial-strength Natural Language Processing (NLP) library. It is used for more complex text analysis, such as **Named Entity Recognition (NER)** to automatically identify Company Names, Locations, or Job Titles from raw text, and lemmatization (converting words to their root form).
- **Sentence Transformers:** Deep learning models that convert text into high-dimensional numerical vectors (embeddings). Instead of just looking for the exact word "Python", transformers understand semantic meaning. This allows the system to match a candidate whose resume says "Backend Developer" with a job asking for a "Server-Side Engineer".

5. Data Preprocessing Steps

Before data can be analyzed or matched, it must be preprocessed:

1. **Text Normalization:** Converting all text to lowercase, stripping whitespace, and removing HTML tags (`<p>`, `<li>`).
2. **Handling Missing Values (Imputation):** Filling in or ignoring fields like missing salaries or locations (e.g. using averages, or NULL).
3. **Tokenization & Stopword Removal:** Breaking text into individual words and removing common filler words (and, the, is) before skill extraction.
4. **Entity Resolution/Deduplication:** Merging duplicate companies or jobs. Ensuring "Amazon" and "Amazon Inc." map to the same company ID.

6. Exploratory Data Analysis (EDA) & Statistics Terminology

Once data is stored, analytics are performed to generate Job Market Reports:

- **Univariate Analysis:** Analyzing a single variable independently.
 - *Example:* A histogram showing the distribution of job postings across different cities, or a count of total jobs requiring "Python".
- **Bivariate Analysis:** Analyzing the relationship between two variables.
 - *Example:* A scatter plot of `min_experience` versus `salary_min` to see if more experience correlates with higher starting pay.
- **Segmented Analysis:** Breaking data down into distinct categories (segments) to compare them.
 - *Example:* Comparing the average salary in the "Data Science" segment versus the "Web Development" segment.

- **Correlation Matrix:** A statistical table showing how closely related multiple numerical variables are (values from -1 to 1).

○ *Example:* Does a higher `job_match_percentage` correlate with a higher `interview_score`?

Other EDA Terms:

- **Mean:** The average value.
- **Median:** The middle value of a dataset. (Often preferred for analyzing salaries, as it's not skewed by a few extremely high-paying outlier jobs).
- **Outliers:** Data points that differ significantly from other observations (e.g., a data entry error showing a \$10,000,000 salary).